

Lab 6: Set Associative Cache Implementation

Assignment

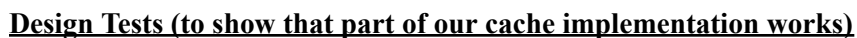
- Add to Pipelined OTTER a two-level data memory:
 - L1 Data Cache, Set Associative, 16 blocks 4 way set associative with 4 words per block (Memory is byte addressable). Write back (so it will need a valid bit and dirty bit) and LRU for the replacement policy
- Test using the same test_all program and evaluate performance difference between Lab 5 and Lab 6. Include the execution results for all four matrices and the test_all program in a comparison table.

Name	Tasks	Contributions
Rodolfo Magana Lopez	Pipeline. Cache. Report.	Most of the System Verilog and Comp Arch design. Lab Report.
Khiem Vu	Pipeline. Cache. Report.	Most of the System Verilog and Comp Arch design. Lab Report
Fiona McGinnis	Lab Report.	Reformat report, attempted cache implementation in Vivado.
Andres Benitez	Synthesizing.	Attempted flashing design on BASYS3.

Overview of Design

- The goal of this lab was to add a set associative cache to our pipelined OTTER.
- We wrote a DMEM_CACHE module that serves as our cache and a FSM_D_Cache module that provides control signals corresponding to different data cache states (ST_READ_DCACHE, ST_READ_DMEM, ST_DMEM_WB)
- We also had to modify our actual D_MEM to accommodate the cache implementation, such as when we dirty miss and other state machine changes.
- In the actual data cache module (DMEM_CACHE), we compacted our sets and ways inside a struct. We utilized nested arrays inside this struct to represent our cache. Inside the struct, there are 4 ways (28 bits each), and 16 words corresponding to that set.
- In DMEM_CACHE, we split the read and write logic into two always_comb blocks. In each of them, we used for loops to iterate through each set, checking for hits or misses.
- In the FSM_D_Cache module, there is a freeze signal that gets set high whenever there is a miss. This freeze carries on to the ST_READ_DMEM and ST_DMEM_WB states. In the main pipeline

- In our current implementation, we couldn't get to the LRU policy. However, we did have a pointer system for each set. Basically, each set pointer will start at 0, and every time we write to a set, the set pointer increments. The location at which the set pointer points to is going to be the location we're replacing for stores. The pointer will then roll over.
- You can see below the hand written notes to set up the Architecture of the cache.



- *Test for filling all ways in set 0 (NO LRU YET)*

```
li x30, 0x600c
li x31, 0xffffffff
sw x31, 0(x30)
li x31, 0x21ab
sb x31, 1(x30)

li x30, 0x7000
li x31, 0xd00d
sw x31, 0(x30)

li x30, 0x800c
li x31, 0x4444
sw x31, 0(x30)

li x30, 0x600c
lw x31, 0(x30)

li x30, 0x800c
```

```

li x31, 0xb0b
sw x31, 0(x30)

li x30, 0x7000
li x31, 0xd00d
sw x31, 0(x30)

li x30, 0x6800
li x31, 0xc477
sw x31, 0(x30)

#Over write set 0, way 0
li x30, 0x9000
li x31, 0x999
sw x31, 0(x30)

li x30, 0x6010 #set 1, way 0
li x31, 0x1010
sw x31, 0(x30)

li x30, 0x6020 #set 2
li x31, 0x2020
sw x31, 0(x30)

li x30, 0x6030 #set 3
li x31, 0x333
sw x31, 0(x30)

li x30, 0x6110 #set 1, way 1
li x31, 0x1010
sw x31, 0(x30)

end: j end

```

For this test we observed in simulation that all the ways in our set 0 had values in them. This demonstrates that our design at this point could handle multiple values getting stored in multiple ways in set 0 (set 0 pointer).

- *Test for filling all ways in all sets (NO LRU YET)*

```

li x30, 0x600c
li x31, 0xffffffff
sw x31, 0(x30)
li x31, 0x21ab
sb x31, 1(x30)

li x30, 0x7000
li x31, 0xd00d
sw x31, 0(x30)

li x30, 0x800c
li x31, 0x4444
sw x31, 0(x30)

li x30, 0x600c
lw x31, 0(x30)

li x30, 0x800c
li x31, 0xb0b
sw x31, 0(x30)

li x30, 0x7000
li x31, 0xd00d
sw x31, 0(x30)

li x30, 0x6800
li x31, 0xc477
sw x31, 0(x30)

```

```

#Over write set 0, way 0
li x30, 0x9000
li x31, 0x999
sw x31, 0(x30)

li x30, 0x6010 #set 1, way 0
li x31, 0x1000
sw x31, 0(x30)

li x30, 0x6020 #set 2, way 0
li x31, 0x2020
sw x31, 0(x30)

li x30, 0x6030 #set 3, way 0
li x31, 0x333
sw x31, 0(x30)

li x30, 0x6110 #set 1, way 1
li x31, 0x1010
sw x31, 0(x30)

li x30, 0x7210 #set 1, way 2
li x31, 0x0102
sw x31, 0(x30)

li x30, 0x8210 #set 1, way 3
li x31, 0x1033
sw x31, 0(x30)

li x30, 0x7020 #set 2, way 1
li x31, 0x2011
sw x31, 0(x30)

li x30, 0x8020 #set 2, way 2
li x31, 0x2022
sw x31, 0(x30)

li x30, 0x8820 #set 2, way 3
li x31, 0x2003
sw x31, 0(x30)

li x30, 0x7730 #set 3, way 1
li x31, 0x33301
sw x31, 0(x30)

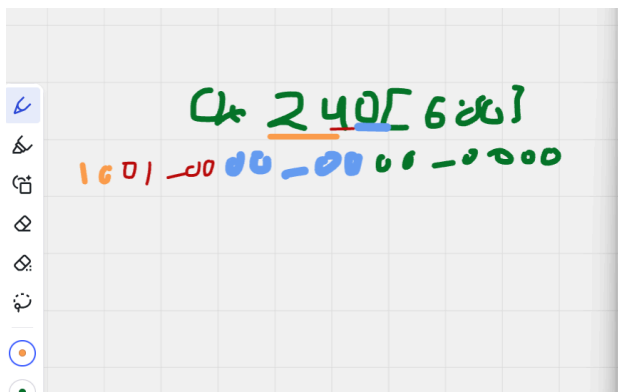
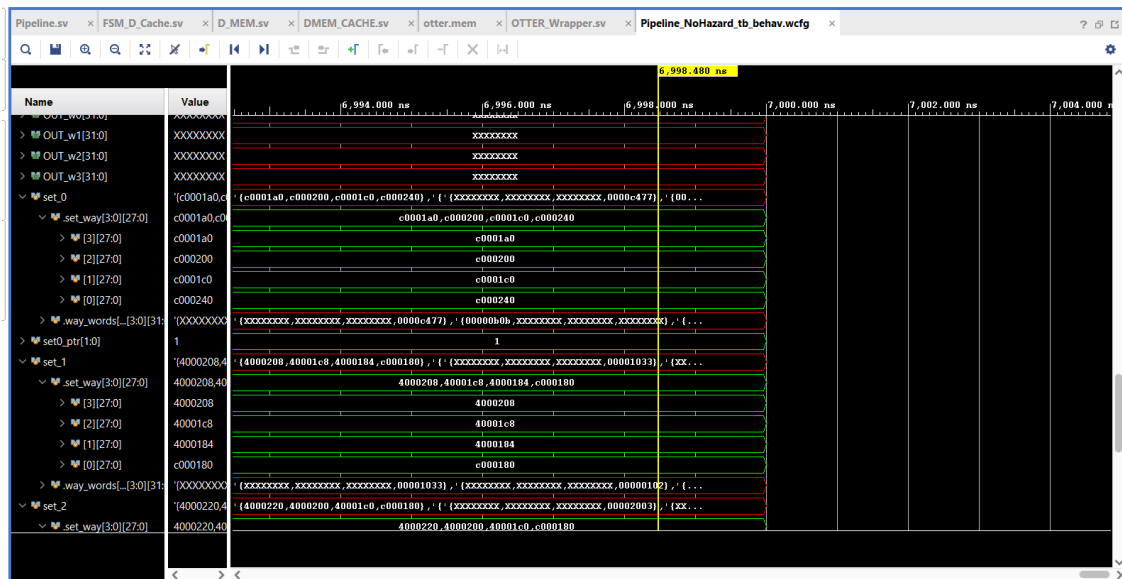
li x30, 0x8530 #set 3, way 2
li x31, 0x33302
sw x31, 0(x30)

li x30, 0x6930 #set 3, way 3
li x31, 0x3333
sw x31, 0(x30)

end: j end

```

SIMULATION RESULTS FOR THIS SPECIFIC TEST IS BELOW



Binary to Hex converter

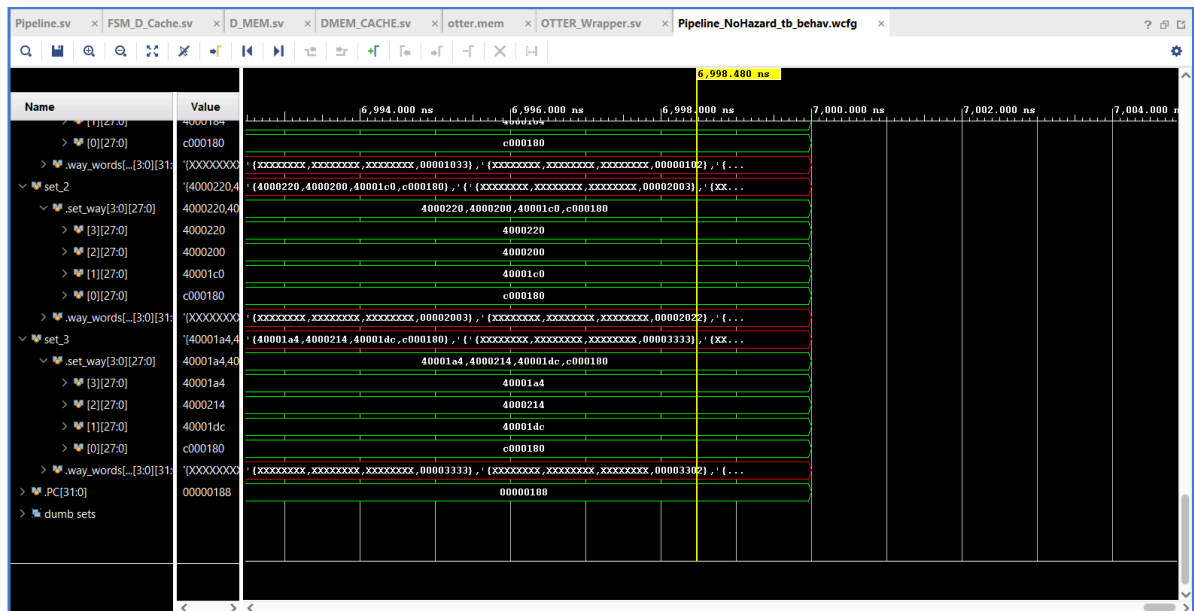
From To

Enter binary number

Hex result

Hex number (4 digits)

The above image shows the binary translation of Set0 Way0. To prove 0x9000 overwrote 0x6000 in way0.

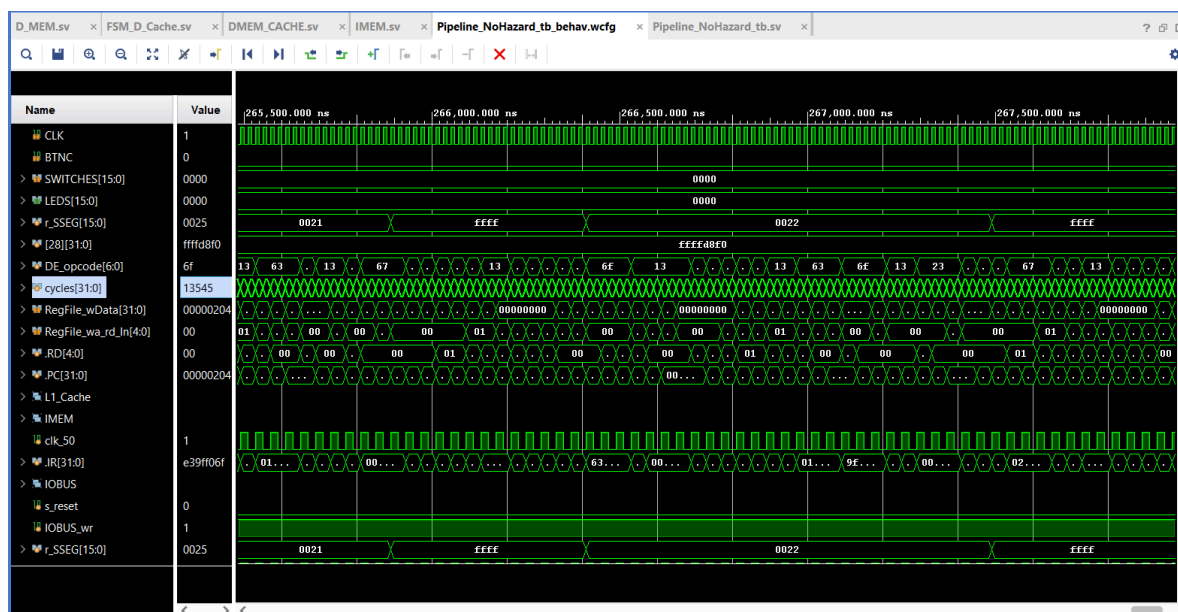
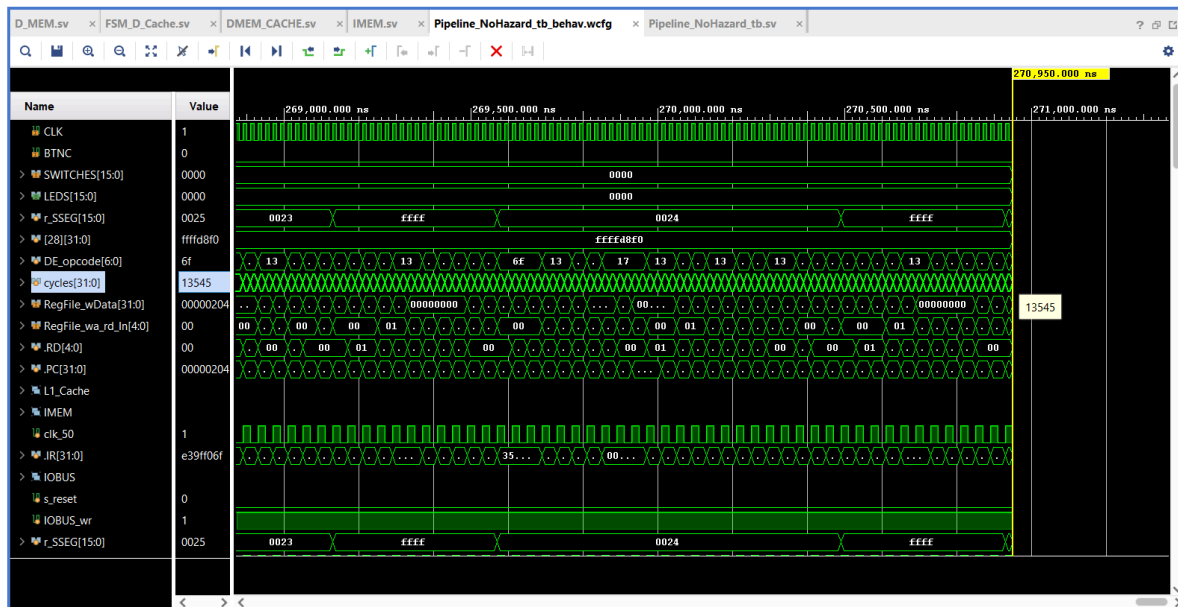


- We can see from the simulation above that our pointer for each set works as intended. It properly fills in the entire block when we write 4 different times, and have even confirmed that when writing a 5th address to a full way, it will overwrite the line that the pointer is pointing to.

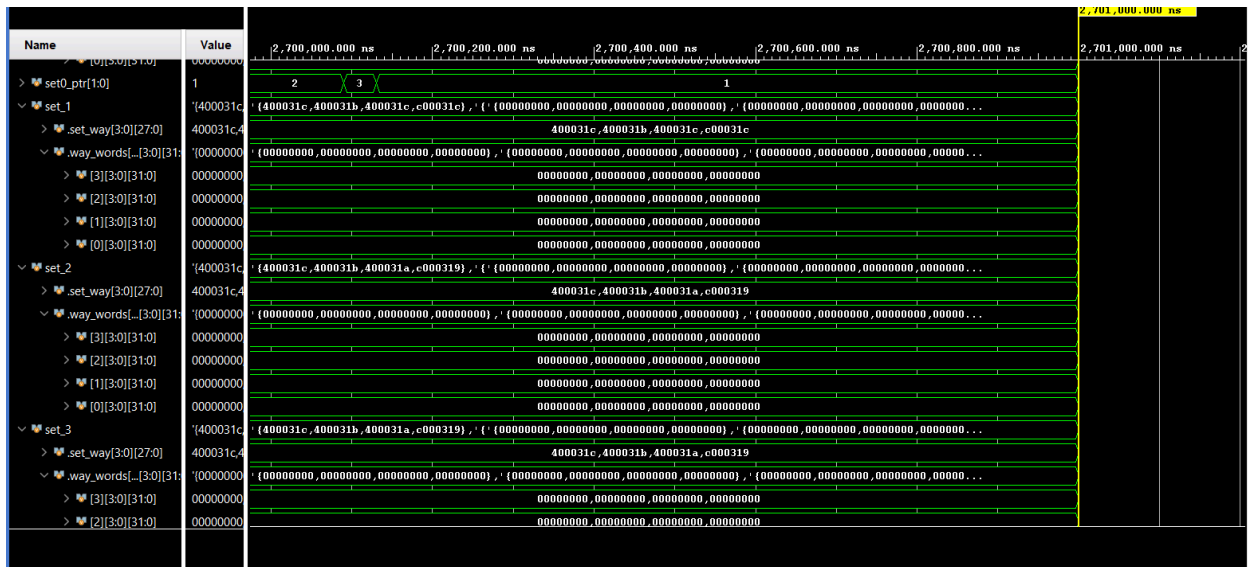
Simulation Results

- **test_all**: Out of all the tests, we failed test 13, 14, 1f, 20, 21, 22, 23, and 24
- **Matrix Multiplication**: Our matrix multiplication did not work.

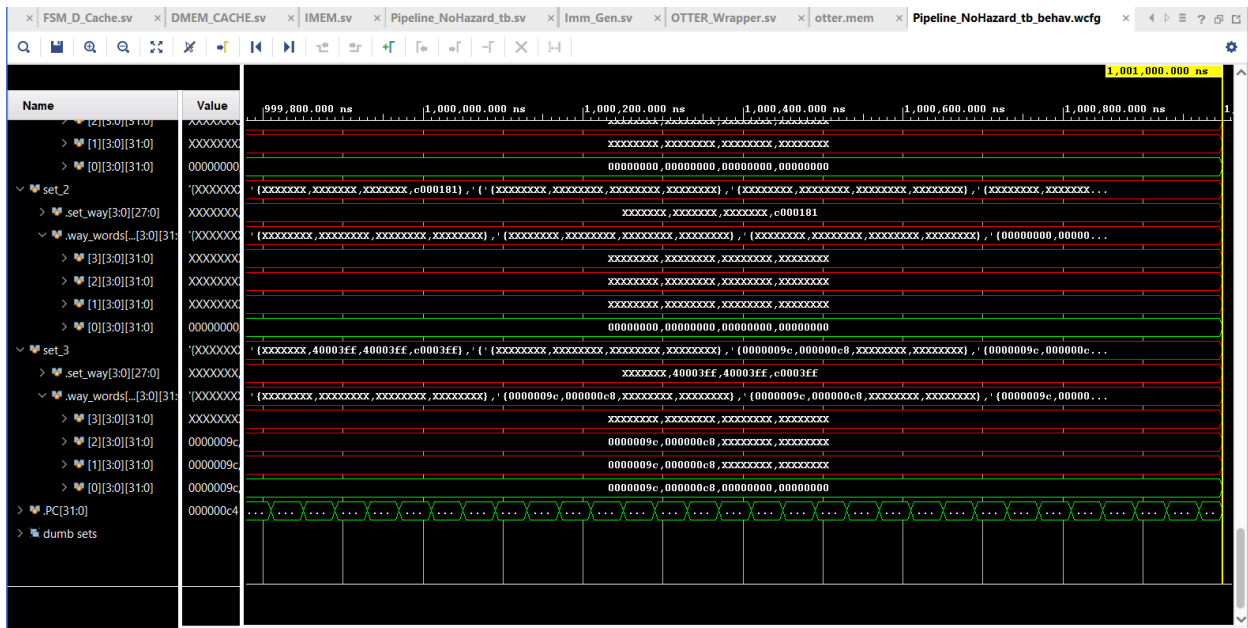
test_all Screenshots



Matrix Mult Screenshots



This screen shot shows that the Ways are getting completely filled because of our Replacement Pointer Policy.



Though on the screenshot above, we can see that the wrong values are being loaded into the cache.

Problems Encountered

- Our pointers in our sets worked properly to overwrite, but we found difficulty in trying to implement even a pseudo LRU method, so we went with a Pointer Policy instead.
- Our Test All fails 8 tests. We believe there is an error either in the pipeline logic or in the Data Cache implementation, but with the complexity of the pipeline, it is very difficult to debug what exactly is going wrong.
- We ran into synthesis issues with this message:
[Synth 8-3391] Unable to infer a block/distributed RAM for 'memory_reg' because the memory

pattern used is not supported. Failed to dissolve the memory into bits because the number of bits (524288) is too large. Use 'set_param synth.elaboration.rodinMoreOptions {rt::set_parameter dissolveMemorySizeLimit 524288}' to allow the memory to be dissolved into individual bits.

- **Because we couldn't synthesize, we couldn't record a video, or get any power/ area information from our design.**
- The above indicates that the memory behavior is not synthesizable since the Basys3 FPGA has BRAM which is dual port. IMEM and DMEM are acting as a multi port memory module which is not how the BRAM in the internal FPGA works. There is not enough space in the FPGA to fit this custom and unrealistic memory access behavior.

Comparisons

PIPELINE W/ HAZARD HANDLING (LAB 4)

Cycles	13,799
Runtime (ns)	276,010

CACHED PIPELINE IMEM ONLY (LAB 5)

Cycles	16,459
Runtime (ns)	329,210

CACHED PIPELINE IMEM and DMEM ONLY (LAB 6)

Cycles	13,545
Runtime (ns)	270,950

Although the test_all program finished earlier, it was probably due to failed tests exiting early and moving on to the next test.